Scale or Shape? Decomposing NTK Movement During Grokking

Group G2
Zain Al-Saffi (s4800836) · Jasper Chong (s4883420) · Aaron D'Mello (s4890257)
Zac Kienzle (s4803349) · Samuel Rieger (s4904959)

Abstract

Grokking is delayed generalisation of a network, but what determines the delay is disputed. We decompose the movement of the neural tangent kernel into its scale and its shape, measuring both on a two-layer network learning modular addition across laziness, width and weight decay. At a fixed width, the shape change is the statistic most tied to the grokking step, even though some networks reach it without grokking. The grokking time grows with width and falls with weight decay, which neither a shape clock nor a weight-decay clock predicts alone. Read early, shape is the best single predictor of the grokking time. We also measure the same statistics on MNIST, where grokking is induced in a different way by scaling up the initial weights. On this task the weight-decay clock holds and width drops out. Weight decay therefore sets the time through the weight norm on both tasks. In our reading, width enters only where the task forces the kernel to change shape.

1 Introduction

Grokking is the delayed generalisation that Power et al. (2022) found on small algorithmic datasets. A network fits its training data early, and its test accuracy rises long afterwards. Modular addition learned by a two-layer perceptron is a small instance of it. Full-batch gradient descent on the squared error groks there with no explicit regularisation (Gromov, 2023). What sets the length of the delay is an open question.

One account holds that grokking is a transition from lazy training to rich training (Kumar et al., 2024). In lazy training the network behaves as a kernel method with its initial tangent kernel. In rich training the kernel adapts to the task. The evidence that the kernel changes is a distance: the empirical tangent kernel stays close to its initial value until the training data are fitted, and moves far from it afterwards (Mohamadi et al., 2024). A distance does not say how the kernel changed. A kernel can grow in scale while its eigenvectors stay fixed. It can also change its shape, so that its eigenvectors change.

The difference matters for the accounts of the grokking time. Recent work times grokking by scale-like parameters: the weight norm (Khanh et al., 2026), the scale of the logits (Khanh, 2026), or the weight-decay rate (Kim, 2026; Pracher et al., 2026). The result that Kumar et al. (2024) cite for weight decay forcing the kernel to change is that of Lewkowycz and Gur-Ari (2021), in which the kernel only shrinks. None of these accounts measures whether the kernel changes shape.

Knowing when a network will grok would be useful in practice. The tangent kernel can be measured on any network trained by gradient descent, while the settings the published clocks need are not always known. An estimate of the time lets training stop once a network has generalised, which saves compute, and it says when during training a capability appears, which an audit of a model has to establish.

We ask what sets the grokking time: the shape of the kernel, its scale, or the weight-decay rate. Our hypothesis is that the shape change sets it. We measure three statistics of the empirical tangent kernel along training: its scale S_t, its shape R_t and its alignment A_t with the labels. We do so on a grid of 720 runs that crosses laziness, width and weight decay. Our contributions are as follows.

1. We split the movement of the kernel into a change of scale and a change of shape, and use each one as a timing variable. On the baseline of Kumar et al. (2024), the usual distance is mostly a change of scale: at $\alpha = 1$ the scale accounts for 90% of D_t^2 at grokking.

2. We test whether grokking happens at a fixed value of each statistic, with a placebo test that reads each run at another run's grokking step, so that a statistic that merely stays in a narrow band is not mistaken for a threshold.
3. We fit a power law for how the grokking time scales with model width and weight decay. The hypotheses that the shape change sets the grokking time and that weight decay sets it predict different exponents.
4. We test whether the statistics can be used to predict grokking ahead of time.
5. We repeat the first two tests on the MNIST dataset, a task on which Liu et al. (2023) induce grokking through a different method, to test if the findings are generalisable.

The main finding is a mixed clock. On modular addition, shape is the most informative of the three statistics, but it is not enough on its own, and its value at grokking changes with width and α. The grokking time depends on both width, which sets the shape change, and weight decay, which shrinks the scale at a fixed width, so neither our shape clock nor the published weight-decay clock holds alone. On MNIST weight decay alone sets the time, through the weight norm, and our own reading, which rests on Mohamadi et al. (2024), is that width enters only where the task forces the kernel to change shape.

2 Related Work

Grokking. Power et al. (2022) report grokking on small algorithmic datasets. Nanda et al. (2023) reverse-engineer the algorithm a small transformer learns for modular addition and split training into memorisation, circuit formation and clean-up. Gromov (2023) gives closed-form weights that solve modular addition with a quadratic activation, and observes grokking under full-batch gradient descent with no explicit regularisation.

Lazy and rich training. In the infinite-width limit the tangent kernel stays constant, so the network trains as a kernel method (Jacot, Gabriel, and Hongler, 2020). Chizat, Oyallon, and Bach (2020) show that lazy training is a matter of scale. Multiplying a model's output by a factor α brings its training closer to that of its linearisation as α grows. Kumar et al. (2024) use this factor to move a network between the two regimes, and propose that grokking is the transition from lazy to rich dynamics. Mohamadi et al. (2024) prove that a permutation-equivariant kernel method needs a constant fraction of the p^2 pairs to learn modular addition, and observe that the kernel moves far from its initial value only after the training data are fitted.

What sets the grokking time. Khanh et al. (2026) find that networks trained with weight decay grok when the weight norm reaches a critical value, and that clamping the norm above that value delays grokking exponentially. Khanh (2026) show that under cross-entropy the norm acts through the scale of the logits, and that under squared error it acts through a different route. Kim (2026, Cor. 5, Eq. 20, arXiv v1) derives, for linear models trained with heavy-ball momentum β and weight decay, a grokking time of about $(1-\beta)/(\eta\lambda)$ times a logarithm of an initial amplitude, and observes this scaling on modular addition. Pracher et al. (2026) predict, for homogeneous networks trained on the squared error with weight decay, that the grokking time is set by the product of the learning rate and the weight decay. Liu et al. (2023) describe the same relationship in terms of the weight norm. When weight decay is the only regulariser and the training loss is near zero after the data are fitted, a network starting with norm w_0 above a critical value w_c generalises after roughly $\ln(w_0/w_c)/(\eta\lambda)$ steps. They induce grokking on MNIST by increasing w_0.

The kernel under weight decay. For a network trained by gradient flow with L_2 strength λ, the kernel evolves as

$$\frac{d\Theta_t(x,x')}{dt} = -2(k-1)\lambda\,\Theta_t(x,x') + T_t(x,x') + T_t(x',x), \tag{1}$$
$$T_t(x,x') = -\sum_{x''} \partial_{\mu\nu} f_t(x)\,\partial_\mu f_t(x')\,\partial_\nu f_t(x'')\,\ell'(x''),$$

where k is the degree of homogeneity of the network, the sum runs over the training inputs x'' and repeated parameter indices, and ℓ' is the derivative of the loss with respect to the output (Lewkowycz and Gur-Ari, 2021, Eqs. S5 and S6, arXiv v2). The weight decay appears only in the first term, which rescales the kernel. Only T_t can change its shape. At initialisation T_0 has expectation of order $1/N$ in the width N, a result that Lewkowycz and Gur-Ari (2021) take from Dyer and Gur-Ari (2020). In the infinite-width limit the kernel therefore only decays, with fixed eigenvectors (Lewkowycz and Gur-Ari, 2021, Thms. 1 and 2).

Predictions. Table 1 sets out what each account predicts for the three tests of this report, on either task. If the shape change sets the grokking time and T_t stays of order $1/N$ along training, a wider network changes shape more slowly and groks later at a fixed weight decay. The weight decay should then matter little. A weight-decay clock (meaning that weight decay sets the grokking time) predicts the opposite. Each account was tested on its own, so these predictions have not been set against each other on one grid. The weight-decay accounts also leave open what sets the time without weight decay. Kim (2026, p. 3, arXiv v1) lists that case, and the creation of new directions by feature learning, among the things its mechanism does not explain. We believe that the shape change of the tangent kernel measures one form of such feature learning.

Table 1: What each account predicts for the three tests. The exponents are those of $\log T = c + a\log N + b\log(\eta\lambda)$. "Not stated" means that the source makes no claim. Entries marked "ours" are our reading of the source. Tier 3 tests only the predictors that can be measured on a network, so it does not test $\eta\lambda$ alone (section 4.4).

Account	Fixed at grokking (Tier 1)	Width exponent a (Tier 2)	Decay exponent b (Tier 2)	Early predictor (Tier 3)
Shape clock (our hypothesis)	R_t reaches a threshold	$a > 0$, since T_t shrinks with width (ours)	$b \approx 0$	R_t
Weight-decay clock (Kim, 2026; Liu et al., 2023; Pracher et al., 2026)	Not stated	$a \approx 0$; width can enter Kim's law only inside the logarithm (ours)	$b \approx -1$	$\eta\lambda$ alone
Norm or scale clock (Khanh, 2026; Khanh et al., 2026)	The weight norm reaches a critical value; in our terms S_t at grokking is fixed (ours)	Not stated	Not stated	S_t

3 Methods

3.1 Task, network and training

The task we train on is modular addition with modulus $p = 23$, at the baseline of Kumar et al. (2024, App. 8.3, arXiv v3). Each pair (a, b) of residues is encoded as the concatenation $x_{a,b} \in \mathbb{R}^{2p}$ of the one-hot codes of a and b. Its target $y_{a,b} \in \mathbb{R}^p$ is the one-hot code of $(a + b) \bmod p$. The dataset holds all $p^2 = 529$ pairs. Each seed draws its own split, with 476 pairs for training and 53 for testing, and its own initial weights.

The network is a two-layer perceptron without biases,

$$f(x;\theta) = N^{-1/2}\, W^{(2)} \phi\big(D^{-1/2} W^{(1)} x\big), \tag{2}$$

with input dimension $D = 2p$, width N, weights $W^{(1)} \in \mathbb{R}^{N\times D}$ and $W^{(2)} \in \mathbb{R}^{p\times N}$ drawn from $\mathcal{N}(0,1)$, and the rectifier $\phi(h) = \max\{h, 0\}$. The factor $N^{-1/2}$ is the NTK parameterisation of Jacot, Gabriel, and Hongler (2020). Following Kumar et al. (2024), the trained predictor is $\tilde{f}(x;\theta) = \alpha[f(x;\theta) - f(x;\theta_0)]$ with learning rate $\eta = \eta_0/\alpha^2$. A larger α gives lazier training. The loss is the squared error averaged over the outputs and the training pairs. Training is full-batch gradient descent with weight decay, $\theta_{k+1} = (1-\eta\lambda)\theta_k - \eta\nabla L(\theta_k)$. For plain gradient descent this is the same as L_2 regularisation (Kim, 2026). Table 10 (Appendix F) lists every setting. The code that trains the runs and rebuilds every figure, table and number is in `report_v2/code/`. **[TODO: Replace with the link to the public repository once it exists.]**

3.2 The grid

Each axis of the grid tests one account. The output scale α sets how lazy the training is (Chizat, Oyallon, and Bach, 2020). The width N sets the size of T_t in (1), and so how fast the kernel can change shape. The weight decay $\eta\lambda$ sets the clock of Kim (2026). We call one combination of $(\alpha, N, \eta\lambda)$ a *cell*. The five runs of a cell share every setting and differ only in their seed, which sets the split and the initial weights. The grid has $3 \times 6 \times 8 = 144$ cells and 720 runs, and every run finished.

3.3 Scale, shape and alignment

We measure the tangent kernel of one scalar output, the sum of the logits divided by $\sqrt{p}$, $g(x;\theta) = p^{-1/2}\sum_c f_c(x;\theta)$. This is the pseudo-NTK of Mohamadi, Bae, and Sutherland (2022). On the 53 test pairs of a seed, the kernel at step t is the matrix with entries $[K_t]_{ij} = \langle \nabla_\theta g(x_i;\theta_t), \nabla_\theta g(x_j;\theta_t)\rangle$. The kernel of $\tilde{f}$ is α^2 times the kernel of f, so the ratios below do not depend on which of the two is used. With Y the one-hot targets of the test pairs and $C = I - \frac{1}{53}\mathbf{1}\mathbf{1}^\top$, we record

$$S_t = \frac{\|K_t\|_F}{\|K_0\|_F}, \qquad R_t = 1 - \frac{\langle K_t, K_0\rangle_F}{\|K_t\|_F\,\|K_0\|_F}, \qquad A_t = \frac{\langle CK_tC,\, CYY^\top C\rangle_F}{\|CK_tC\|_F\,\|CYY^\top C\|_F}. \tag{3}$$

The scale S_t is 1 at initialisation, and a value above 1 means the kernel grew. The shape R_t measures how far the kernel has changed shape, and it is the kernel distance of Fort et al. (2020). It is 0 at initialisation and does not change when the kernel is rescaled. The alignment A_t is the centred alignment of Cortes, Mohri, and Rostamizadeh (2024) with the label kernel $YY^\top$, and it also does not change when the kernel is rescaled.

The usual measure of kernel movement is the relative variation $D_t = \|K_t - K_0\|_F/\|K_0\|_F$ (Geiger et al., 2020). It is a function of the other two,

$$D_t^2 = S_t^2 - 2S_t(1 - R_t) + 1, \tag{4}$$

(appendix A). A kernel that only grows has $R_t = 0$ and $D_t = |S_t - 1|$. A large D_t therefore does not show that the kernel changed shape.

3.4 Event times

The memorisation step of a run is the first checkpoint at which its training accuracy reaches 0.99. Its grokking step T is the first checkpoint at which its test accuracy reaches 0.95. Appendix E repeats the analysis at the levels 0.80 and 0.90. Both events are counted from step 0. A run that does not reach a level within 200 000 steps is kept, and its grokking step is recorded as censored at the budget (200 000 steps). The events are read on the checkpoint grid described above, so each one is known to lie between two checkpoints. The gap to the checkpoint before is at most 9.7% of the grokking step and 10.9% of the memorisation step.

3.5 Analysis

Thresholds (Tier 1). For each statistic we take its value at the grokking step of every run that groks, and measure the spread across runs by the standard deviation (SD). A statistic can have a small spread because it stays in a narrow band all through training, so we include a placebo test to separate the two cases. The placebo test reads each run at the grokking step of another run, drawn by a random permutation with no fixed points, and measures the SD again. The SD ratio is the median placebo SD over 2 000 permutations divided by the SD at the true events, and p is the share of permutations whose SD is no larger. A ratio near 1 means the statistic is not tied to the event. The SD carries the units of its statistic, so the SDs of different statistics cannot be compared directly.

Scaling (Tier 2). We fit the log-normal accelerated failure time model

$$\log T = c + a \log N + b \log(\eta\lambda) + \gamma \log N \log(\eta\lambda) + \sigma\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0,1), \tag{5}$$

by maximum likelihood. A censored run contributes the probability that its grokking step exceeds the budget (Kalbfleisch and Prentice, 2002). Both logarithms are centred at the middle of the grid, so a and b are the slopes there. The fits use lifelines (Davidson-Pilon, 2019). The intervals resample whole cells and then seeds within cells, so that they include the misfit of the cells about the fitted plane.

Prediction (Tier 3). We read S_t, $\log R_t$ and A_t at $t^* = 2\,430$, a saved step before any run reaches the grokking level, and ask how well they predict the grokking step. At t^* the values of R_t cluster near zero. 46% of the runs that grok have R_t below 0.01, and the largest is 0.13. Taking the logarithm spreads them out and predicts better, so we use $\log R_t$. Kernel ridge on all three statistics reaches 0.47 with R_t and 0.55 with $\log R_t$. The five seeds of a cell are close copies, so whole groups of runs are always held out. Two tests are used. The first holds out one cell at a time, which tests interpolation. Kernel ridge regression with a Gaussian kernel is fitted on each subset of the statistics, which shows which statistics carry the information. The second holds out one value of α at a time, which tests extrapolation to an unseen laziness. It compares a linear accelerated failure time model on each statistic alone, gradient-boosted trees with the accelerated failure time objective (Chen and Guestrin, 2016) on all three, and a gated recurrent unit (Cho et al., 2014) that reads all three at the 25 saved steps up to t^*. The weight decay does not enter as a predictor, since it is a setting and not a measurement (section 4.4). As a reference, the second test also fits the trees to the settings N and $\eta\lambda$ together and to each one alone. Two scores are used. R^2 is the share of the spread in $\log T$ explained on held-out

runs. The concordance index is the share of pairs of runs that a model puts in the right order, so that 0.5 is a coin toss (Harrell et al., 1982). Appendix D gives the settings of each model.

3.6 The second task

In Tier 4 we ask the questions of Tiers 1 and 2 on MNIST, in the setup with which Liu et al. (2023) induce grokking. The network is trained on 1 000 images, and its initial weights are multiplied by a scale $s > 1$, so weight decay must shrink the norm before the network generalises. Section 3.1 instead subtracts $f(x; \theta_0)$, so that every run starts from the zero function. On MNIST this centring removes the delay, since the training and test accuracies then rise together, so we do not use it. The architecture is a bias-free three-layer rectifier network of width N in the standard parameterisation, trained by minibatch gradient descent without momentum, with the learning rate reset from the curvature every 10000 steps. The values tested on the grid are $s \in \{3, 5, 7, 9\}$, $N \in \{100, 200, 400, 800\}$ and seven decays from 0 to 4×10^{-5}, each with 5 seeds. Each run has a budget of 500000 steps and the grokking level is 0.80 since no run reaches 0.95 (Appendix G).

4 Experiments

4.1 Baseline: the reproduction of Kumar et al.

Finding. Our runs reproduce the baseline of Kumar et al. (2024). The network memorises within a few thousand steps and groks much later, and a lazier network groks later. Over this time the kernel grows several-fold while R_t stays below 0.09.

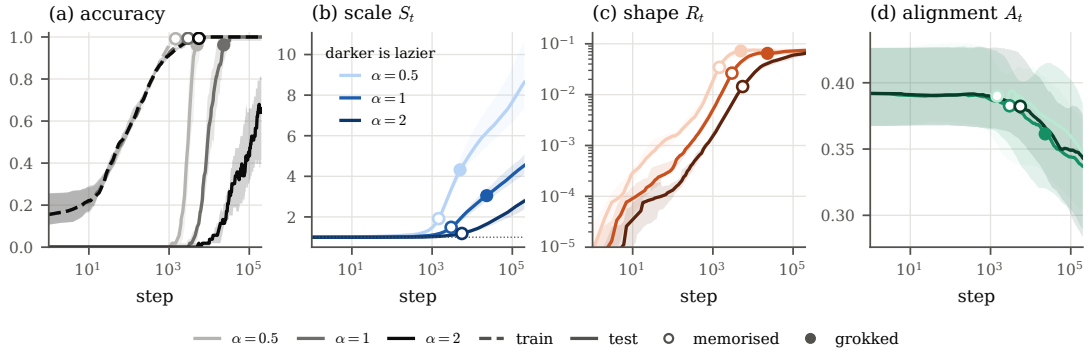


Figure 1: Training at the baseline of Kumar et al. (2024) ($N = 100$, $\eta\lambda = 0$) for three values of α. Each line is the median over 5 seeds and the shading is their range. Darker lines are lazier networks. A hollow marker is the median memorisation step and a filled marker the median grokking step. No seed groks at $\alpha = 2$. R_t is on a log scale.

At $\alpha = 0.5$ and $\alpha = 1$ every seed memorises and then groks, with median grokking steps of 4 981 and 23 000 (Figure 1). At $\alpha = 2$ no seed groks within 200 000 steps. These grokking steps and their order in α are close to those of Kumar et al. (2024, Fig. 2(c), arXiv v3). Appendix B compares the settings and the event steps one by one.

By (4) the squared variation splits into a scale part $(S_t - 1)^2$ and a shape part $2S_tR_t$. At $\alpha = 1$ the scale part is 90% of D_t^2 at grokking. The distance D_t thus mostly records the growth of the kernel, which is why we measure S_t and R_t separately.

4.2 Tier 1: is there a threshold?

Finding. At a fixed width, shape is the statistic most tied to the grokking step, but reaching that shape change does not make a network grok. The alignment at grokking barely differs from its value at initialisation. Across widths, no statistic takes a fixed value at grokking.

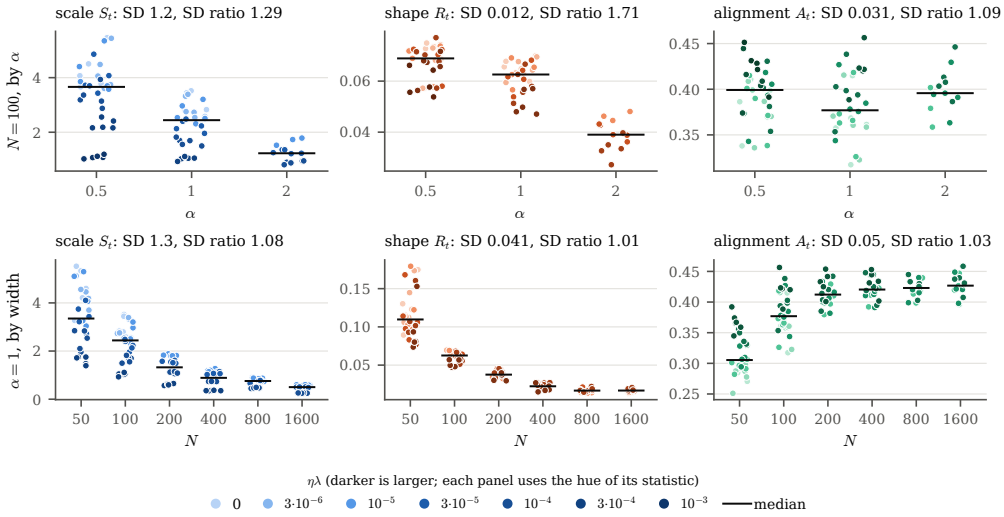


Figure 2: The kernel statistics at the grokking step of every run that groks. Top row: $N = 100$, grouped by α. Bottom row: $\alpha = 1$, grouped by width. Colour is $\eta\lambda$, darker for larger, and bars are medians. Each title gives the SD at grokking and the SD ratio of the placebo test (Table 2).

Table 2: Spread of each statistic across runs that grok, as a standard deviation, at the grokking step, at the memorisation step and at another run's grokking step (placebo, median over 2 000 permutations). The SD ratio is the placebo SD over the SD at grokking, and p is the share of permutations with an SD no larger than the true one.

Runs	Statistic	At grokking	At memorisation	Placebo	SD ratio	p
$N = 100$ (78 runs)	S_t	1.2	0.42	1.6	1.29	< 0.0005
	R_t	0.012	0.0094	0.02	1.71	< 0.0005
	A_t	0.031	0.02	0.034	1.09	0.029
$\alpha = 1$ (130 runs)	S_t	1.3	0.46	1.4	1.08	0.258
	R_t	0.041	0.035	0.041	1.01	0.423
	A_t	0.05	0.033	0.052	1.03	0.079

At $N = 100$, R_t is the statistic most tied to the event. Its SD widens 1.71-fold when each run is read at another run's grokking step, against 1.29-fold for S_t and 1.09-fold for A_t (Table 2). No permutation gave R_t or S_t a spread as small as the true one. The alignment tightens only a little ($p = 0.029$). Its median over the runs that grok is 0.398 at initialisation and 0.406 at grokking. Across widths no statistic passes the placebo test at level 0.95. At $\alpha = 1$ the ratios are 1.08, 1.01 and 1.03, and no p is below 0.05. At grokking R_t falls from 0.108 at $N = 50$ to 0.019 at $N = 1600$, and S_t falls from 4.11 to 0.58 (medians over all α and $\eta\lambda$). At level 0.80 (appendix E) the scale and the alignment tighten slightly across widths, and at 0.90 only the alignment does. R_t does not at any level. Pooled over all 355 runs that grok, S_t tightens 1.69-fold ($p < 0.0005$). The grokking steps of that pool span nearly two decades while S_t keeps growing, so this may reflect growth more than a threshold.

Reaching the shape change at which runs grok is not enough to grok. There are 34 cells in which some seed memorises but no seed groks. In 8 of them, at least one seed reaches the median R_t at grokking of its width before the budget runs out. One of them is the baseline cell at $\alpha = 2$ (section 4.1).

Wider networks grok with a smaller shape change but take longer, which suggests that width sets the rate at which the kernel changes shape. Section 4.3 tests this. Our proposal expected the alignment to cross a threshold shared by every cell, and that does not hold. Other claims of a fixed value at grokking, such as the critical weight norm of Khanh et al. (2026), can be given the same placebo check.

4.3 Tier 2: width or weight decay?

Finding. The grokking time depends on both width and weight decay, so neither the shape clock nor the weight-decay clock of Table 1 holds on its own. Width sets how much the kernel changes shape. At a fixed width, weight decay shrinks its scale.

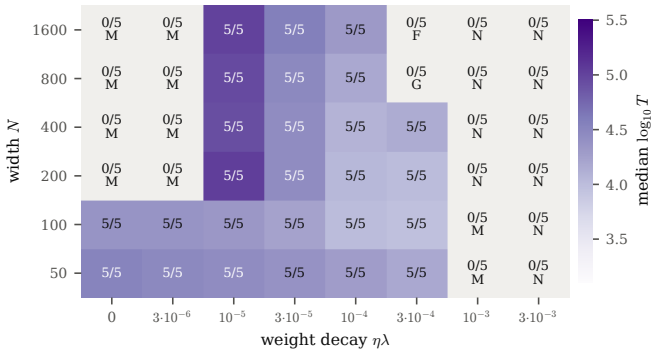


Figure 3: Grokking time and outcome over width and weight decay at $\alpha = 1$. Colour is the median $\log_{10} T$ over the seeds that grok, shown when at least 3 of 5 grok. Each square is one cell and gives the seeds that grok out of 5. A cell in which none grok also shows the most common phase at the last step: M memorises only, F forgets, N never fits, and G ends above test accuracy 0.9 without reaching 0.95.

Without weight decay, only $N = 50$ and $N = 100$ grok within the budget (Figure 3). The wider networks memorise and stop there. From $\eta\lambda = 10^{-5}$ to 10^{-4} every seed at every width groks, and the time falls with $\eta\lambda$ at every width. The largest weight decay at which every seed groks is 3×10^{-4} for $N \leq 400$ and 10^{-4} for $N = 800$ and $N = 1600$. At 10^{-3} and above no width groks. A wider network thus groks later at the same weight decay and stops grokking at a smaller one, so the weight decay that speeds grokking depends on the width.

Table 3: Fits of (5) at $\alpha = 1$ beside the predictions of Table 1. Brackets give 95% intervals from a bootstrap that resamples whole cells and then seeds within cells. The main row uses the 90 runs with $10^{-5} \le \eta\lambda \le 10^{-4}$, where every seed at every width groks. The other rows are checks.

Fit	a	b	γ	σ
Shape clock predicts	> 0	≈ 0		
Weight-decay clock predicts	≈ 0	≈ -1		
Main range, level 0.95	$+0.24$ $[+0.08, +0.40]$	-0.63 $[-0.92, -0.46]$	-0.16 $[-0.37, +0.05]$	0.34
Adding $\eta\lambda = 3 \times 10^{-4}$	$+0.36$ $[+0.03, +0.62]$	-0.21 $[-0.60, +0.14]$	$+0.14$ $[-0.24, +0.48]$	0.80
Level 0.80	$+0.30$ $[+0.17, +0.45]$	-0.55 $[-0.74, -0.39]$	-0.17 $[-0.36, -0.02]$	0.29
Level 0.90	$+0.27$ $[+0.13, +0.41]$	-0.60 $[-0.84, -0.42]$	-0.16 $[-0.37, +0.03]$	0.31
Memorisation step	-1.53 $[-1.80, -1.32]$	$+0.07$ $[-0.13, +0.31]$	$+0.05$ $[-0.23, +0.29]$	0.38

Both exponents are nonzero, and both intervals exclude zero ($a = +0.24$, $b = -0.63$). A wider network groks later, as the shape clock predicts, and more weight decay shortens the time, as the weight-decay clock predicts. Neither exponent takes the value its account predicts. Wider networks memorise sooner, so the width effect on grokking is not a delay in fitting the training data (memorisation row). The estimate of γ is negative, which would mean that weight decay shortens the time more at large width. Its interval includes zero, so the data do not fix its sign. Adding the cells at $\eta\lambda = 3 \times 10^{-4}$, where the widest networks stop grokking, changes the fit: b moves to -0.21 and its interval includes zero. The exponents therefore hold for the range in which every seed groks.

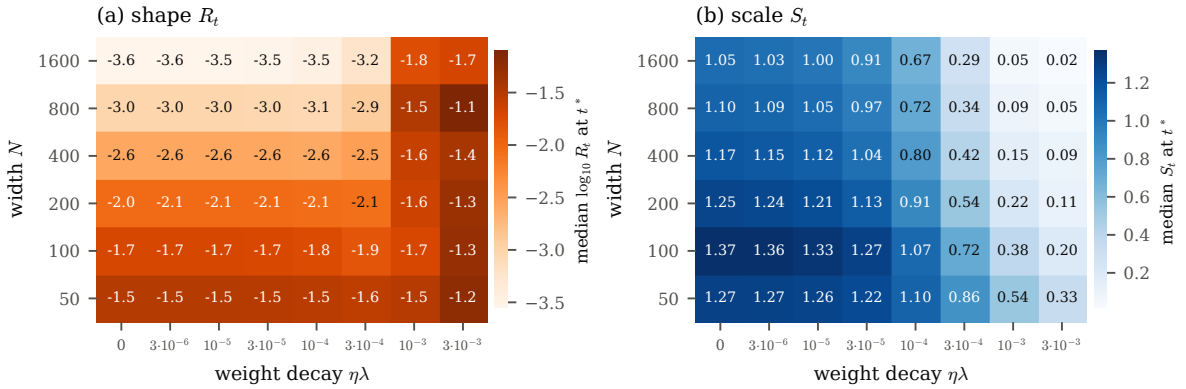


Figure 4: Width sets the shape change of the kernel, and weight decay shrinks its scale. Medians over seeds at $\alpha = 1$, read at $t^* = 2\,430$, before any run groks: (a) $\log_{10} R_t$ and (b) S_t.

Up to $\eta\lambda = 3 \times 10^{-4}$, the shape change at t^* depends on width and hardly on weight decay (Figure 4). Narrow networks change shape the most. The scale falls with weight decay at every width. Among the runs that grok, width alone explains 0.66 of the spread of $\log R_t$, and weight decay 0.18. The spread of S_t is shared: α explains 0.28, width 0.22 and weight decay 0.26. This links Tier 2 back to Tier 1. Width sets the shape change, weight decay is one of the settings that set the scale, and the grokking time depends on both.

4.4 Tier 3: predicting the grokking time from the kernel

Finding. Read before any run groks, shape is the best single predictor of the grokking time on held-out cells. Scale adds information that shape lacks, and alignment adds little. On an unseen

α, the three statistics together rank runs better than any one of them alone. The settings N and $\eta\lambda$ rank them better still, and better than either setting alone, which supports the mixed clock of section 4.3.

Each account in Table 1 names a different early predictor: R_t for the shape clock, S_t for the norm or scale clock, and $\eta\lambda$ alone for the weight-decay clock. This tier tests the first two, with A_t beside them. It does not test $\eta\lambda$, which we set and know exactly, while a general network reports neither its effective regularisation nor its laziness. Its tangent kernel can be measured. We want predictors that carry over to real models, so this tier asks how much of the grokking time the kernel statistics recover on their own. A model given the settings N and $\eta\lambda$ serves as a reference that knows more than a general network would report.

Table 4: Kernel ridge regression on subsets of the statistics at t^*, for the 355 runs that grok, scored with each of the 74 cells held out in turn. The last column is the share of the spread of each quantity that lies between cells.

Statistics	R^2	Concordance	Quantity	Between cells
A_t	0.05	0.55	$\log T$	0.96
S_t	0.27	0.67	S_t	0.99
$\log R_t$	0.35	0.70	$\log R_t$	0.99
S_t and $\log R_t$	0.51	0.76	A_t	0.56
$\log R_t$ and A_t	0.39	0.72		
S_t and A_t	0.23	0.65		
All three	0.55	0.78		

Taken alone, $\log R_t$ explains the most of the grokking time on held-out cells (Table 4). Adding S_t to $\log R_t$ raises R^2 from 0.35 to 0.51, so shape does not carry all of the information. Adding A_t to both raises it only to 0.55. Between 0.96 and 0.99 of the spread of $\log T$, S_t and $\log R_t$ lies between cells (Table 4). The statistics are therefore almost fully set by the cell, and they say nothing about which seed of a cell groks first.

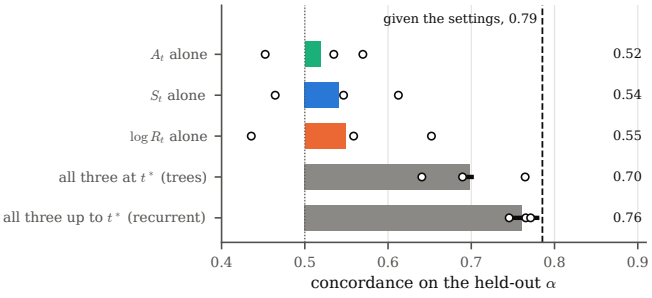


Figure 5: Predicting the grokking step of runs with an unseen α. Each model is fitted on two values of α and scored on the third by the concordance index with censoring. Each statistic alone enters a linear accelerated failure time model. The trees read all three statistics at t^*, and the recurrent network reads them at the 25 saved steps up to t^*. Bars are the mean over the three held-out values and dots each value. Black lines give the range over seeds (5 for the trees, 3 for the recurrent network). The dashed line is the trees given the settings N and $\eta\lambda$.

On an unseen α each statistic alone ranks runs only a little better than a coin toss (Figure 5): 0.52 for A_t, 0.54 for S_t and 0.55 for $\log R_t$. Gradient-boosted trees on all three reach 0.70, and the recurrent network that reads their history up to t^* reaches 0.76. The reference model given

the settings reaches 0.79, more than any model of the kernel statistics. Nanda et al. (2023, p. 1, arXiv v3) call a progress measure a metric that varies smoothly, precedes the phase transition and is causally linked to it. Shape meets the first two conditions here. We have not tested a causal link, and reaching a given shape change does not make a network grok (section 4.2). Shape also gives no earlier or better warning than the settings themselves.

The settings reference combines width and weight decay. The same trees given $\log N$ alone reach 0.67, and given $\eta\lambda$ alone they reach 0.72 (0.71 to 0.73 over seeds). Each setting alone therefore ranks runs worse than the two together. This supports the mixed clock of section 4.3, in which the grokking time depends on both width and weight decay. It also sets the limit of the use described in the Introduction: the statistics time a run from quantities that any network exposes, while a network whose settings are known is timed better from them.

4.5 Tier 4: a second task, MNIST digits

Finding. On MNIST the weight-decay clock of Table 1 holds exactly. The grokking time falls inversely proportional to weight decay and does not depend on width. At grokking, the weight norm reaches a fixed value, and the kernel shrinks with it while its shape remains nearly unchanged.

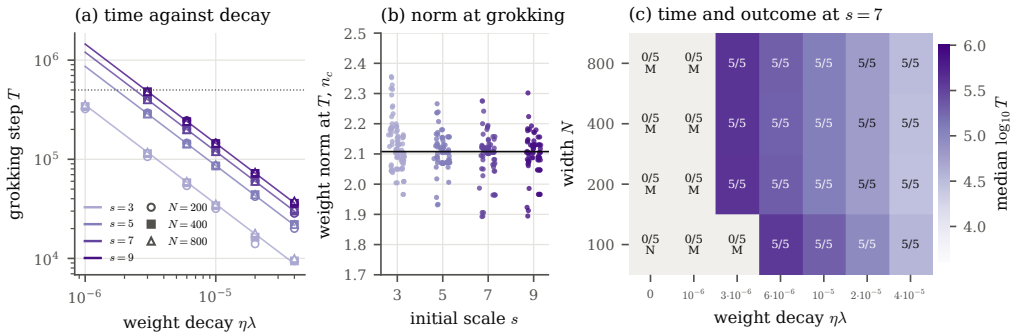


Figure 6: MNIST at $s = 7$. Panels (a) and (b) use widths 200 to 800; (c) adds width 100, which reaches the level later and off the law (Appendix G). (a) The grokking step against weight decay, medians over 5 seeds, coloured by the initial scale s with one marker per width. The lines are $T = \ln(s/n_c)/(\eta\lambda)$ with the mean n_c of panel (b), and the dotted line is the budget. (b) The weight norm at grokking in units of the norm at $s = 1$, one point per run, with its mean. (c) The map of Figure 3 at $s = 7$: the median grokking step over the seeds where at least 3 of 5 grok, the seeds that grok out of 5, and the phase letter of Figure 3 where none does.

In the main grid range, $s = 7$ at $N \in \{200, 400, 800\}$ with $\eta\lambda$ from 3×10^{-6} to 10^{-5}, every seed groks, and the fit of (5) gives $a = -0.02$ [-0.07, $+0.02$] and $b = -1.00$ [-1.05, -0.96], against $+0.24$ and -0.63 on modular addition (Table 3), with a residual σ of 0.03 against 0.34.

This is likely a direct result of the weight norm. The product $\eta\lambda T$ is constant within each scale and increases with s (Figure 6a), and at grokking the norm is 2.11 times the norm at $s = 1$, with a standard deviation of 0.07 (3%) over the 314 runs that reach the level, across every scale, width and decay (Figure 6b). As in Liu et al. (2023) the norm follows pure decay and the time is given by the law $T = \ln(s/n_c)/(\eta\lambda)$ (Appendix G). As the norm falls, the kernel shrinks too, and at grokking it is between 0.0012 and 0.0050 of its initial value. As on modular addition, the shape change decreases with width. On MNIST it falls sharply, from 0.045 at $N = 200$ to 0.002 at $N = 800$, while T remains approximately the same. The Tier 1 placebo test cannot distinguish these statistics here (Appendix G).

The two tasks therefore grok by different routes. Grokking in MNIST is set by the Liu et al. (2023) problem construction where the enlarged initial function must decay and the kernel need only shrink before generalisation occurs. On modular addition the delay happens because of the centring, and Mohamadi et al. (2024) prove that a fixed permutation-equivariant kernel cannot learn the task from a constant fraction of the pairs. In our reading, the kernel must change shape first, at a rate that depends on width.

5 Limitations

Every modular addition run uses the NTK parameterisation, one task and one architecture. A width effect measured here belongs to this parameterisation. Each cell has 5 seeds. The events are known only to the resolution of the checkpoint grid. The fits in Tier 2 depend on which cells enter, and the decay edge bends the plane (Table 3). The kernel statistics say nothing about which seed of a cell groks first, and the reference model given the settings predicts better than any kernel model. The alignment here uses the kernel of the summed logits. The tangent kernel of all p logits may behave differently. The centred predictor is not homogeneous, so the theorem behind (1) applies to our network only as a guide. The proposal also planned a clamped-norm arm, a sweep over p and the training fraction, and a cross-entropy arm. They were not run. The proposal defined the events by the loss. The report uses accuracy (section 3.4).

The MNIST runs differ in not only the task but the architecture and method for inducing grokking, with the most impactful of these being the uncentred predictor. Our comparison assumes that none of these changes which clock applies. With only two tasks to compare we cannot tell which of them, if any, is responsible for the different result. Two small tasks can show that a clock fails, as the width exponent does on MNIST, and they cannot fix the exponents for other architectures, losses or scales. Tier 4 tests which clock sets the time once a delay exists, not whether a shape change can cause one.

6 Conclusion

We asked what sets the grokking time: the shape of the tangent kernel, its scale, or the weight-decay rate. On modular addition the answer is a mixed clock. Shape is the most informative of the three statistics. At a fixed width it is the most tied to the grokking step, and read early it is the best single predictor of the time. It is not enough on its own, and it is not a fixed threshold. Networks can reach the shape change at which runs grok without grokking, its value at grokking changes with width and α, and early scale adds information that shape lacks. The grokking time grows with width and falls with weight decay, so neither our shape clock nor the weight-decay clock holds alone. Width sets the shape change, and at a fixed width weight decay shrinks the scale. The two settings together also predict the time better than either one alone. On MNIST, Liu et al. (2023) create the delay by enlarging the initial weights. There the time follows the law $\ln(s/n_c)/(\eta\lambda)$ of the weight-decay clock, and width sets the shape change without setting the time. Weight decay therefore sets the grokking time through the weight norm on both tasks. On modular addition width sets it as well, while on MNIST only the weight-decay route acts. Our own reading, which rests on Mohamadi et al. (2024), is that width enters only where a fixed kernel cannot solve the task, so that the kernel must change shape first. What is new is the split into scale and shape, the placebo test, the crossed grid on two tasks, and prediction from quantities that any network exposes. A next step is to test whether the early rate of shape change falls as $1/N$, and whether a kernel frozen at each step can generalise on either task.

Contribution

[TODO: One line for each member, from the group.]

- Al-Saffi: **[TODO:]**
- Chong: **[TODO:]**
- D'Mello: **[TODO:]**
- Kienzle: **[TODO:]**
- Rieger: **[TODO:]**

Acknowledgement

[TODO: Anyone outside the group who advised on the project.]

Use of AI tools

[TODO: The task sheet asks each deliverable to state how AI tools were used. The group should agree on this text. A draft follows.] The group used an AI assistant (Claude, by Anthropic) to draft parts of the code, to restructure this report, and to check references against their arXiv pages. Group members checked every result and every reference before submission.

[TODO: Check whether the appendices have a page limit. The copy of `project.pdf` in the repository gives none, and the marking sheet should be read again. If the limit is 5 pages, about half a page has to go: on the build of 4 November 2026 the appendices run from the lower half of page 13 to the lower part of page 18. The candidates, in order of least loss: the prose of Appendix G, parts of which repeat Table 11; the last paragraph of Appendix E on seed-only intervals; and Appendix B, which would free three quarters of a page on its own and is the evidence for the reproduction.]

A The identity for the variation

Expanding the squared norm gives $\|K_t - K_0\|_F^2 = \|K_t\|_F^2 - 2\langle K_t, K_0\rangle_F + \|K_0\|_F^2$. Dividing by $\|K_0\|_F^2$ and using $\langle K_t, K_0\rangle_F = (1 - R_t)\|K_t\|_F\|K_0\|_F$ from (3) gives $D_t^2 = S_t^2 - 2S_t(1 - R_t) + 1$. By the Cauchy-Schwarz inequality, $R_t = 0$ exactly when $K_t = cK_0$ for some $c > 0$, and then $D_t = |S_t - 1|$.

B The baseline against Kumar et al.

Table 5 sets each setting of our baseline beside Kumar et al. (2024, arXiv v3), read from their Appendices 8.1 and 8.3, Section 3 and Figure 2, and Table 6 sets our event steps beside the steps read by eye from their Figure 2. Their grokking at $\alpha = 0.5$ and $\alpha = 1$ falls close to ours. Their training accuracy rises much more slowly, reaching 100% near 20 000 epochs at $\alpha = 1$, where our runs reach 0.99 by about 3 000 steps. They do not state the parameterisation or the initialisation, which may explain this gap. The reproduction therefore matches on the grokking time and on the order in α, and does not match on the memorisation time.

Section 4.1 gives the split of the variation at grokking for $\alpha = 1$. The scale part $(S_t - 1)^2$ is 76% of D_t^2 at memorisation and 95% at the last step, where S_t is 4.5 and R_t is only 0.074.

At grokking it is 94% for $\alpha = 0.5$, and for $\alpha = 2$ it is 48% at memorisation and 90% at the last step.

Table 5: The baseline setting by setting. "Where" is the place in Kumar et al. (2024, arXiv v3) that gives the setting.

Setting	Kumar et al. (2024)	This report	Where
Task	$a + b \bmod p$, $p = 23$	$a + b \bmod p$, $p = 23$	App. 8.3
Encoding	two one-hot codes, $D = 2p$	two one-hot codes, $D = 2p$	App. 8.3
Output	p outputs, one-hot target	p outputs, one-hot target	App. 8.3
Training pairs	90% of p^2	90% of p^2 (476 of 529)	App. 8.3
Width	$N = 100$	$N = 100$ (grid: 50 to 1600)	App. 8.3
Network	one hidden layer	one hidden layer, no biases, rectifier	App. 8.3
Parameterisation	not stated	NTK, weights from $\mathcal{N}(0, 1)$	
Loss	mean squared error	mean squared error	Sec. 3
Optimiser	vanilla full-batch GD, no weight decay	full-batch GD; $\eta\lambda = 0$ at the baseline	App. 8.3
Predictor	$\alpha[f(x;\theta) - f(x;\theta_0)]$	the same	App. 8.1
Learning rate	$\eta = 100$; $\eta = \eta_0/\alpha^2$	$\eta = 100/\alpha^2$	App. 8.1, 8.3
α	$\{0.5, 1, 1.5, 2\}$	$\{0.5, 1, 2\}$	Fig. 2(c)
Budget	100 000 epochs (Fig. 2(c))	200 000 steps	Fig. 2(c)
Seeds	one run shown	5 per cell	Fig. 2

Table 6: Event steps at $N = 100$ and $\eta\lambda = 0$. The values of Kumar et al. (2024) are read by eye from their Fig. 2(a) and (c), so they are approximate.

	Kumar et al. (2024), read from Fig. 2		This report, median [range] over 5 seeds	
α	Train 100%	Test 100%	Memorisation	Grokking (0.95)
0.5	≈4 000	≈5 000	1 455 [1 313, 1 612]	4 981 [3 662, 6 776], 5/5
1	≈20 000 to 25 000	≈25 000 to 27 000	2 983 [2 692, 2 983]	23 000 [15 395, 34 000], 5/5
1.5	≈65 000	≈70 000	not in the grid	
2	≈90 000	about 25% at 100 000	5 520 [4 496, 6 000]	0/5 by 200 000

C Outcomes over the whole grid

Table 7 gives the outcome of every cell. An entry gives the seeds that grok at level 0.95 out of 5. After a colon it gives the median grokking step in thousands, when at least 3 seeds grok. A cell in which no seed groks shows 0 and the most common phase at the last step, as in Figure 3. Every seed groks in 35 of the 48 cells at $\alpha = 0.5$, 26 at $\alpha = 1$ and 8 at $\alpha = 2$.

Table 7: Outcomes over the whole grid at level 0.95: seeds that grok out of 5, and the median grokking step in thousands.

α	N	0	3×10^{-6}	10^{-5}	3×10^{-5}	10^{-4}	3×10^{-4}	10^{-3}	3×10^{-3}
0.5	50	5: 7	5: 7	5: 8	5: 7	5: 7	5: 5	5: 6	0 N
	100	5: 5	5: 5	5: 4	5: 4	5: 4	5: 4	5: 4	0 N
	200	3: 27	5: 26	5: 14	5: 7	5: 5	5: 4	5: 6	0 N
	400	0 M	5: 70	5: 19	5: 9	5: 6	5: 4	0 M	0 N
	800	0 M	5: 106	5: 33	5: 12	5: 7	5: 6	0 N	0 N
	1600	0 M	5: 157	5: 48	5: 17	5: 8	5: 9	0 N	0 N
1	50	5: 33	5: 30	5: 27	5: 24	5: 19	5: 15	0 M	0 N
	100	5: 23	5: 23	5: 21	5: 17	5: 10	5: 9	0 M	0 N
	200	0 M	0 M	5: 106	5: 27	5: 11	5: 10	0 N	0 N
	400	0 M	0 M	5: 82	5: 27	5: 12	5: 14	0 N	0 N
	800	0 M	0 M	5: 88	5: 30	5: 15	0 G	0 N	0 N
	1600	0 M	0 M	5: 99	5: 37	5: 20	0 F	0 N	0 N
2	50	0 M	0 M	1	2	1	0 M	0 N	0 N
	100	0 M	0 M	3: 95	5: 69	5: 43	0 M	0 N	0 N
	200	0 M	0 M	0 M	5: 86	5: 40	0 M	0 N	0 N
	400	0 M	0 M	0 M	5: 83	5: 60	0 N	0 N	0 N
	800	0 M	0 M	0 M	5: 94	0 G	0 N	0 N	0 N
	1600	0 M	0 M	0 M	5: 98	0 M	0 N	0 N	0 N

D Settings of the prediction models

All models use the 630 runs with $\eta\lambda \leq 10^{-3}$, with S_t, $\log R_t$ and A_t read at t^*. The grokking step of a censored run is known only to exceed 200 000. Whole groups are held out: one value of α at a time (Figure 5) or one cell at a time (Table 4). The file `report_v2/code/models.py` fixes every setting below.

Linear models. The log-normal accelerated failure time model of a single statistic standardises it on the training fold and uses a ridge penalty of 0.01.

Kernel ridge. The kernel is Gaussian on standardised inputs. Its penalty in $\{0.01, 0.1, 1\}$ and its width in $\{0.1, 0.5, 2\}$ are chosen by grouped 5-fold cross-validation over the training cells, inside every outer fold. It is fitted to $\log T$ on the training runs that grok.

Gradient-boosted trees. The trees use the accelerated failure time objective with normal errors of scale 1 and a subsample of 0.8. The search covers depths 1 to 4, learning rates 0.03 and 0.1, and minimum child weights 1, 5 and 10, with up to 1 000 rounds and early stopping after 50. The setting with the best concordance on 20% of the training cells is refitted on all training runs. The seeds are 0 to 4. The reference model is the same search on $\log N$ and $\log_{10}(\eta\lambda)$, with $\eta\lambda = 0$ set to 10^{-6}. Its two parts are the same search on $\log N$ alone and on $\log_{10}(\eta\lambda)$ alone.

Recurrent network. One gated recurrent layer with 16 units reads A_t, S_t and R_t at the 25 saved steps up to t^*. R_t enters without a logarithm because it is 0 at step 0. The inputs are standardised on the training fold. A linear head gives the location and a learned parameter the scale of a log-normal accelerated failure time loss. Training uses Adam with learning rate 0.01 and weight decay 10^{-4}, on the full batch, for up to 800 epochs. Every 10 epochs the loss on 20%

of the training cells is checked, and the best weights are kept. Early stopping chose between 70 and 350 epochs. The seeds are 0 to 2.

E Sensitivity to the grokking level

Table 8 repeats the placebo test at the levels 0.80 and 0.90, and Table 9 repeats the main fit. At $N = 100$ the order of the statistics is the same at every level. R_t tightens most, with ratios of 1.83 at 0.80 and 1.79 at 0.90, and S_t comes next. Across widths R_t never passes. At 0.80 the scale and the alignment tighten slightly across widths, with p of 0.024 and 0.001 and ratios of only 1.25 and 1.08. At 0.90 only the alignment does ($p = 0.013$); the scale has $p = 0.085$. The exponents a and b keep their signs at every level, and both intervals exclude zero.

Table 8: The placebo test of Table 2 at three grokking levels. The number of runs that grok is in brackets.

Level	Runs	Statistic	SD at grokking	Placebo SD	Ratio	p
0.80	$N = 100$ (86)	S_t	1.1	1.59	1.45	0.000
		R_t	0.0122	0.0223	1.83	0.000
		A_t	0.0291	0.033	1.13	0.002
0.80	$\alpha = 1$ (136)	S_t	1.01	1.26	1.25	0.024
		R_t	0.0376	0.0391	1.04	0.131
		A_t	0.0446	0.048	1.08	0.001
0.90	$N = 100$ (79)	S_t	1.18	1.58	1.34	0.000
		R_t	0.0118	0.021	1.79	0.000
		A_t	0.0307	0.0335	1.09	0.029
0.90	$\alpha = 1$ (134)	S_t	1.15	1.35	1.17	0.085
		R_t	0.0392	0.0403	1.03	0.195
		A_t	0.0475	0.0501	1.06	0.013
0.95	$N = 100$ (78)	S_t	1.25	1.6	1.29	0.000
		R_t	0.0119	0.0203	1.71	0.000
		A_t	0.0313	0.0341	1.09	0.029
0.95	$\alpha = 1$ (130)	S_t	1.33	1.44	1.08	0.258
		R_t	0.0408	0.0411	1.01	0.423
		A_t	0.0504	0.0521	1.03	0.079

Table 9: The main fit of Table 3 at three grokking levels, with intervals that resample cells and seeds.

Level	a	b	γ	σ
0.80	+0.30 [+0.17, +0.45]	−0.55 [−0.74, −0.39]	−0.17 [−0.36, −0.02]	0.29
0.90	+0.27 [+0.13, +0.41]	−0.60 [−0.84, −0.42]	−0.16 [−0.37, +0.03]	0.31
0.95	+0.24 [+0.08, +0.40]	−0.63 [−0.92, −0.46]	−0.16 [−0.37, +0.05]	0.34

The intervals in Table 3 resample cells as well as seeds. The misfit of the cells about the fitted plane has an SD of 0.32 in $\log T$, against 0.11 for the spread between the seeds of one cell. An interval that resamples seeds alone holds the cells fixed and is much narrower: for a it runs from +0.21 to +0.26.

F Settings of the modular addition runs

Table 10: Every setting of the runs. One combination of $(\alpha, N, \eta\lambda)$ is a cell (section 3.2), and each cell has 5 seeds.

Setting	Value
Modulus, pairs	$p = 23$, all 529 pairs; 476 for training and 53 for testing
Split and seeds	Model seed $s \in \{0, \ldots, 4\}$; the split uses data seed $42 + s$
Network	Equation. (2), no biases, rectifier, weights from $\mathcal{N}(0, 1)$
Width N	$\{50, 100, 200, 400, 800, 1600\}$
Output scale α	$\{0.5, 1, 2\}$
Learning rate	$\eta = \eta_0/\alpha^2$ with $\eta_0 = 100$
Weight decay $\eta\lambda$	$\{0, 3 \times 10^{-6}, 10^{-5}, 3 \times 10^{-5}, 10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$
Optimiser and loss	Full-batch gradient descent, no momentum; mean squared error
Budget	200 000 steps
Checkpoints	307 per run: step 0, 120 log-spaced steps to 200 000, and every 1 000 steps
Saved weights	48 steps per run
Runs	720, all finished
Compute	18.9 h summed over runs; 2.7 h elapsed on 12 workers of one Mac

G Settings of the MNIST runs

Table 11 gives every setting of the MNIST sweep beside the modular addition setting it replaces. Table 12 gives the fit of section 4.5 with its check rows, and the exponents are the same in every row. The runs of one width and one scale were trained together as one fused model on one H100, in 134 summed GPU hours, with TF32 matrix multiplies. A second sweep in full float32 gave the same grokking outcome in every one of the 560 runs and the same memorisation outcome in all but 3.

Some outcomes stay out of section 4.5. Width 100 reaches the level in 87 of its 120 runs with decay, but its test accuracy at memorisation is already 0.72, so there is little delay to time; it reaches the level later than the wider networks at the same decay, with a lower and scattered norm, so it does not follow the law and is left out of the fits. We read this as a limit of capacity at 1 000 images. Without weight decay no run reaches the level within the budget. A network at $s = 3$ already has test accuracy 0.52 when it memorises. The norm follows pure decay, $\|\theta_T\| = \|\theta_0\| e^{-\eta\lambda T}$, to within 0.008 of the initial norm, and the law of section 4.5 has correlation 0.9963 with the measured product $\eta\lambda T$. The alignment at grokking is 0.396 against 0.421 at initialisation, as on modular addition. Under the placebo every statistic tightens, because each is a function of the step and the grokking steps span more than a decade: the ratios are 1.29 to 8.06 at $N = 200$ and 1.30 to 47.54 across widths. Tier 3 is not repeated: with σ of 0.03, an early read of the norm predicts the time and the shape does not. The centred pilot of section 3.6 used the two-layer network of section 3.1 on MNIST with initial weights times 3, full-batch gradient descent, $\eta\lambda = 10^{-5}$, one seed and 200 000 steps, at $\alpha \in \{1, 0.5, 0.05, 0.01\}$. Every run memorised within 1 500 steps with test accuracy between 0.87 and 0.90, and the run at $\alpha = 0.001$ did not memorise.

Table 11: The MNIST runs setting by setting, beside the modular addition setting of Table 10. The learning-rate rule and the probe follow `mnist/README.md`.

Setting	MNIST	Modular addition
Task	1 000 training images drawn by data seed $42 + s$; all 10 000 test images	476 of 529 pairs, split by seed
Network	Three layers of width N, rectifier, no biases, standard parameterisation, uniform initial weights multiplied by s	Equation. (2), NTK parameterisation
Predictor	Uncentred, $f(x;\theta)$	Centred, $\alpha[f - f_0]$
Laziness	Initial scale $s \in \{3, 5, 7, 9\}$	$\alpha \in \{0.5, 1, 2\}$
Width N	$\{100, 200, 400, 800\}$	$\{50, \ldots, 1600\}$
Weight decay $\eta\lambda$	$\{0, 10^{-6}, 3\times10^{-6}, 6\times10^{-6}, 10^{-5}, 2\times10^{-5}, 4\times10^{-5}\}$, fixed per step	eight values from 0 to 3×10^{-3}
Optimiser and loss	SGD, batches of 200, no momentum; mean squared error on one-hot targets	Full-batch gradient descent
Learning rate	$\eta_0 = 1/\lambda_0$ with λ_0 the top Gauss-Newton eigenvalue at initialisation; reset every 10 000 steps to $\eta_0\lambda_0/\lambda_t$, capped at $1000\,\eta_0$	η_0/α^2, fixed
Budget and checkpoints	500 000 steps; 120 log-spaced steps and every 2 000 steps	200 000 steps; 307 checkpoints
Kernel and statistics	Closed-form kernel of the summed logits over $\sqrt{10}$, on a fixed draw of 1 000 test images	Kernel of the summed logits over $\sqrt{p}$, on the 53 test pairs
Events	Memorisation at 0.99; grokking at test accuracy 0.80, counted from step 0; censored at 500 000	Grokking at 0.95; censored at 200 000
Runs	560, all finished; 134 summed GPU hours on H100s	720

Table 12: Fits of (5) on MNIST at $N \in \{200, 400, 800\}$, with 95% intervals from the bootstrap of Table 3. The main row is the fit quoted in section 4.5: $s = 7$ with $\eta\lambda$ from 3×10^{-6} to 10^{-5}, where every seed groks. The other rows change one thing.

Fit	a	b	γ	σ
Main, $s = 7$, level 0.80 (45 runs)	-0.02 [-0.07, $+0.02$]	-1.00 [-1.05, -0.96]	$+0.01$ [-0.07, $+0.10$]	0.03
$s = 5$	-0.02 [-0.06, $+0.03$]	-1.00 [-1.04, -0.96]	$+0.01$ [-0.07, $+0.09$]	0.03
$s = 9$	-0.02 [-0.05, $+0.02$]	-1.00 [-1.02, -0.97]	$+0.00$ [-0.06, $+0.06$]	0.02
Adding $\eta\lambda$ up to 4×10^{-5}	-0.02 [-0.06, $+0.01$]	-1.00 [-1.01, -0.99]	$+0.03$ [-0.00, $+0.06$]	0.03
Level 0.85 (1 censored)	-0.08 [-0.15, -0.02]	-1.00 [-1.06, -0.94]	$+0.00$ [-0.11, $+0.15$]	0.03
Memorisation step	$+0.08$ [-0.02, $+0.15$]	-0.12 [-0.25, -0.04]	$+0.00$ [-0.21, $+0.23$]	0.09

References

Chen, T. and C. Guestrin (2016). *XGBoost: A Scalable Tree Boosting System*. DOI: 10.48550/arXiv.1603.02754. arXiv: 1603.02754 [cs.LG].

Chizat, L., E. Oyallon, and F. Bach (Jan. 2020). *On Lazy Training in Differentiable Programming*. DOI: 10.48550/arXiv.1812.07956. arXiv: 1812.07956 [math.OC].

Cho, K. et al. (2014). *Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation*. DOI: 10.48550/arXiv.1406.1078. arXiv: 1406.1078 [cs.CL].

Cortes, C., M. Mohri, and A. Rostamizadeh (Apr. 2024). *Algorithms for Learning Kernels Based on Centered Alignment*. DOI: 10.48550/arXiv.1203.0550. arXiv: 1203.0550 [cs.LG].

Davidson-Pilon, C. (Aug. 2019). "Lifelines: Survival Analysis in Python". In: *Journal of Open Source Software* 4.40, p. 1317. ISSN: 2475-9066. DOI: 10.21105/joss.01317.

Dyer, E. and G. Gur-Ari (2020). "Asymptotics of Wide Networks from Feynman Diagrams". In: *International Conference on Learning Representations*. DOI: 10.48550/arXiv.1909.11304. arXiv: 1909.11304 [cs.LG].

Fort, S. et al. (Oct. 2020). *Deep Learning versus Kernel Learning: An Empirical Study of Loss Landscape Geometry and the Time Evolution of the Neural Tangent Kernel*. DOI: 10.48550/arXiv.2010.15110. arXiv: 2010.15110 [cs.LG].

Geiger, M. et al. (Nov. 2020). "Disentangling Feature and Lazy Training in Deep Neural Networks". In: *Journal of Statistical Mechanics: Theory and Experiment* 2020.11, p. 113301. ISSN: 1742-5468. DOI: 10.1088/1742-5468/abc4de.

Gromov, A. (Jan. 2023). *Grokking Modular Arithmetic*. DOI: 10.48550/arXiv.2301.02679. arXiv: 2301.02679 [cs.LG].

Harrell Jr., F. E. et al. (1982). "Evaluating the Yield of Medical Tests". In: *JAMA* 247.18, pp. 2543–2546. DOI: 10.1001/jama.1982.03320430047030.

Jacot, A., F. Gabriel, and C. Hongler (Feb. 2020). *Neural Tangent Kernel: Convergence and Generalization in Neural Networks*. DOI: 10.48550/arXiv.1806.07572. arXiv: 1806.07572 [cs.LG].

Kalbfleisch, J. D. and R. L. Prentice (2002). *The Statistical Analysis of Failure Time Data*. 2nd ed. Wiley Series in Probability and Statistics. Wiley. ISBN: 978-0-471-36357-6. DOI: 10.1002/9781118032985.

Khanh, T. X. (June 2026). *What Does the Weight Norm Control in Grokking? Logit-scale Mediation under Cross-Entropy*. DOI: 10.48550/arXiv.2606.18465. arXiv: 2606.18465 [cs.LG].

Khanh, T. X. et al. (June 2026). *The Weight Norm Sets the Grokking Timescale: A Causal Delay Law*. DOI: 10.48550/arXiv.2606.13753. arXiv: 2606.13753 [cs.LG].

Kim, T. (July 2026). *Grokking on the Weight-Decay Clock: A Rate Hierarchy from Softly Broken Symmetries*. DOI: 10.48550/arXiv.2607.23967. arXiv: 2607.23967 [cs.AI].

Kumar, T. et al. (Apr. 2024). *Grokking as the Transition from Lazy to Rich Training Dynamics*. DOI: 10.48550/arXiv.2310.06110. arXiv: 2310.06110 [stat.ML].

Lewkowycz, A. and G. Gur-Ari (Jan. 2021). *On the Training Dynamics of Deep Networks with l_2 Regularization*. DOI: 10.48550/arXiv.2006.08643. arXiv: 2006.08643 [stat.ML].

Liu, Z., E. J. Michaud, and M. Tegmark (Mar. 2023). *Omnigrok: Grokking beyond Algorithmic Data*. DOI: 10.48550/arXiv.2210.01117. arXiv: 2210.01117 [cs.LG].

Mohamadi, M. A., W. Bae, and D. J. Sutherland (2022). *A Fast, Well-Founded Approximation to the Empirical Neural Tangent Kernel*. DOI: 10.48550/arXiv.2206.12543. arXiv: 2206.12543 [stat.ML].

Mohamadi, M. A. et al. (July 2024). *Why Do You Grok? A Theoretical Analysis of Grokking Modular Addition*. DOI: 10.48550/arXiv.2407.12332. arXiv: 2407.12332 [cs.LG].

Nanda, N. et al. (Oct. 2023). *Progress Measures for Grokking via Mechanistic Interpretability*. DOI: 10.48550/arXiv.2301.05217. arXiv: 2301.05217 [cs.LG].

Power, A. et al. (Jan. 2022). *Grokking: Generalization beyond Overfitting on Small Algorithmic Datasets*. DOI: 10.48550/arXiv.2201.02177. arXiv: 2201.02177 [cs.LG].

Pracher, L. et al. (Sept. 2026). *A Spectral Theory of Grokking: Weight Decay Induces Feature Learning*. DOI: 10.48550/ARXIV.2609.26679. arXiv: 2609.26679 [cs.LG].

Indicate whether all your group members consent for your report to be used as a teaching resource by including one of the statement below.

We give consent for this to be used as a teaching resource.
We DO NOT give consent for this to be used as a teaching resource.

[TODO: Keep one of the two statements.]

[TODO: Before submission, confirm that the course report template is `proposal_template.tex`, which the preamble of this file follows word for word, and that the font size, the page size and every line marked "Don't change this" are unchanged.]